

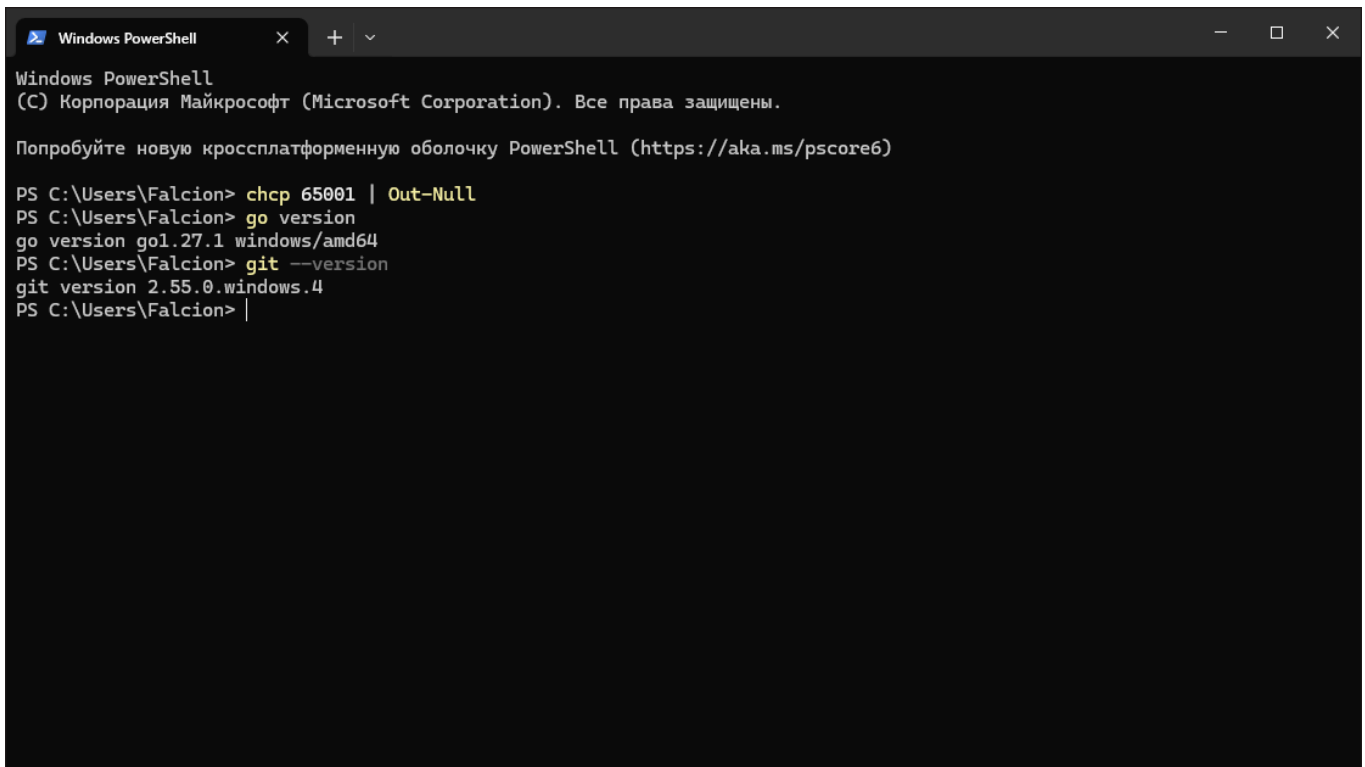
Подготовка окружения и проверка версий

В первую очередь включаем кодировку UTF-8, чтобы в консоли корректно отображался кириллический вывод:

```
chcp 65001 | Out-Null
```

Проверяем, что инструменты Git и CLI для Go доступны:

```
go version  
git --version
```



```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Попробуйте новую кроссплатформенную оболочку PowerShell (https://aka.ms/pscore6)

PS C:\Users\Falcion> chcp 65001 | Out-Null
PS C:\Users\Falcion> go version
go version go1.27.1 windows/amd64
PS C:\Users\Falcion> git --version
git version 2.55.0.windows.4
PS C:\Users\Falcion> |
```

Структура проекта и инициализация модуля

Выбираем рабочую папку. Создаём проект и инициализируем модуль:

```
mkdir helloapi  
cd helloapi  
go mod init example.com/helloapi
```

```
Windows PowerShell
Mode                LastWriteTime         Length Name
----                -
d-----          25.09.2026    17:46          T1.1

PS D:\.WORKSPACE\Education> cd D:\.WORKSPACE\Education\T1.1
PS D:\.WORKSPACE\Education\T1.1> mkdir helloapi

Каталог: D:\.WORKSPACE\Education\T1.1

Mode                LastWriteTime         Length Name
----                -
d-----          25.09.2026    17:46          helloapi

PS D:\.WORKSPACE\Education\T1.1> cd helloapi
PS D:\.WORKSPACE\Education\T1.1\helloapi> go mod init example.com/helloapi
go: creating new go.mod: module example.com/helloapi
PS D:\.WORKSPACE\Education\T1.1\helloapi> ls

Каталог: D:\.WORKSPACE\Education\T1.1\helloapi

Mode                LastWriteTime         Length Name
----                -
-a-----          25.09.2026    17:47          39 go.mod
```

Создаём структуру каталогов:

```
mkdir cmd\server
```

```
Windows PowerShell
PS D:\.WORKSPACE\Education\T1.1\helloapi> tree /f
Folder PATH listing for volume Второстепенный диск
Volume serial number is 0879-1073
D:..
|  go.mod
|_ cmd
   |_ server
PS D:\.WORKSPACE\Education\T1.1\helloapi>
```

Минимальный HTTP-сервер (первая версия)

Создаём файл `cmd\server\main.go` и вставляем код:

```

package main

import (
    "encoding/json"
    "fmt"
    "log"
    "net/http"
)

type user struct {
    ID    string `json:"id"`
    Name string `json:"name"`
}

func main() {
    mux := http.NewServeMux()

    // Текстовый ответ
    mux.HandleFunc("/hello", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "Hello, world!")
    })

    // Пока временный JSON (без UUID — добавим на шаге 4)
    mux.HandleFunc("/user", func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "application/json")
        _ = json.NewEncoder(w).Encode(user{
            ID:    "temp",
            Name:  "Gopher",
        })
    })

    addr := ":8080"
    log.Printf("Starting on %s ...", addr)
    log.Fatal(http.ListenAndServe(addr, mux))
}

```

Подключение внешней зависимости и доработка /user

Подтягиваем пакет для генерации UUID:

```

go get github.com/google/uuid@latest
go mod tidy

```

```
Windows PowerShell
PS D:\.WORKSPACE\Education\T1.1\helloapi> tree /f
Folder PATH listing for volume Второстепенный диск
Volume serial number is 0879-1073
D:..
├── go.mod
└── cmd
    └── server

PS D:\.WORKSPACE\Education\T1.1\helloapi> go get github.com/google/uuid@latest
go: downloading github.com/google/uuid v1.6.0
go: added github.com/google/uuid v1.6.0
PS D:\.WORKSPACE\Education\T1.1\helloapi> go mod tidy
PS D:\.WORKSPACE\Education\T1.1\helloapi> ls

Каталог: D:\.WORKSPACE\Education\T1.1\helloapi

Mode                LastWriteTime         Length Name
----                -
d-----         25.09.2026   17:48             cmd
-a-----         25.09.2026   17:53             39 go.mod
-a-----         25.09.2026   17:53              0 go.sum

PS D:\.WORKSPACE\Education\T1.1\helloapi> |
```

Правим `cmd\server\main.go`: добавляем импорт `github.com/google/uuid` и заменяем `"temp"` на реальный UUID:

```
mux.HandleFunc("/user", func(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    _ = json.NewEncoder(w).Encode(user{
        ID:    uuid.NewString(), // теперь реальный UUID
        Name:  "Gopher",
    })
})
```

Запуск сервера и быстрая проверка

Из корня проекта запускаем сервер:

```
go run ./cmd/server
```

Запускаем сервер:

```
PS D:\.WORKSPACE\Education\T1.1\helloapi> go mod tidy
go: finding module for package github.com/google/uuid
go: found github.com/google/uuid in github.com/google/uuid v1.6.0
PS D:\.WORKSPACE\Education\T1.1\helloapi> go run ./cmd/server
2026/09/25 18:02:39 Starting on :8080 ...
|
```

Проверяем эндпоинты:

```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Попробуйте новую кроссплатформенную оболочку PowerShell (https://aka.ms/pscore6)

PS C:\Users\Falcion> curl http://localhost:8080/hello

StatusCode      : 200
StatusDescription : OK
Content         : Hello, world!
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 13
                  Content-Type: text/plain; charset=utf-8
                  Date: Fri, 25 Sep 2026 15:02:53 GMT

                  Hello, world!
Forms           : {}
Headers         : {[Content-Length, 13], [Content-Type, text/plain; charset=utf-8], [Date, Fri, 25 Sep 2026 15:02:53 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 13

PS C:\Users\Falcion> curl http://localhost:8080/user
```

```
Windows PowerShell
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 13

PS C:\Users\Falcion> curl http://localhost:8080/user

StatusCode      : 200
StatusDescription : OK
Content         : {"id":"5e1a5301-0b10-4155-a677-f38e60adfc5f","name":"Gopher"}
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 62
                  Content-Type: application/json
                  Date: Fri, 25 Sep 2026 15:02:55 GMT

                  {"id":"5e1a5301-0b10-4155-a677-f38e60adfc5f","name":"Gopher"}
Forms           : {}
Headers         : {[Content-Length, 62], [Content-Type, application/json], [Date, Fri, 25 Sep 2026 15:02:55 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 62

PS C:\Users\Falcion> |
```

Сборка бинарника и повторная проверка

Собираем проект в бинарник:

```
go build -o helloapi.exe ./cmd/server
```

Запускаем собранный бинарник:

```
PS D:\.WORKSPACE\Education\T1.1\helloapi> go build -o helloapi.exe ./cmd/server
PS D:\.WORKSPACE\Education\T1.1\helloapi> .\helloapi.exe
2026/09/25 18:05:47 Starting on :8080 ...
```

Проверяем эндпоинты аналогичным способом:

```
Windows PowerShell
RawContentLength : 62

PS C:\Users\Falcion> curl http://localhost:8080/hello

StatusCode      : 200
StatusDescription : OK
Content          : Hello, world!
RawContent       : HTTP/1.1 200 OK
                  Content-Length: 13
                  Content-Type: text/plain; charset=utf-8
                  Date: Fri, 25 Sep 2026 15:06:39 GMT
                  Hello, world!
Forms           : {}
Headers         : {[Content-Length, 13], [Content-Type, text/plain; charset=utf-8], [Date, Fri, 25 Sep 2026 15:06:39 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 13

PS C:\Users\Falcion> curl http://localhost:8080/user
```

```
Windows PowerShell
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 13

PS C:\Users\Falcion> curl http://localhost:8080/user

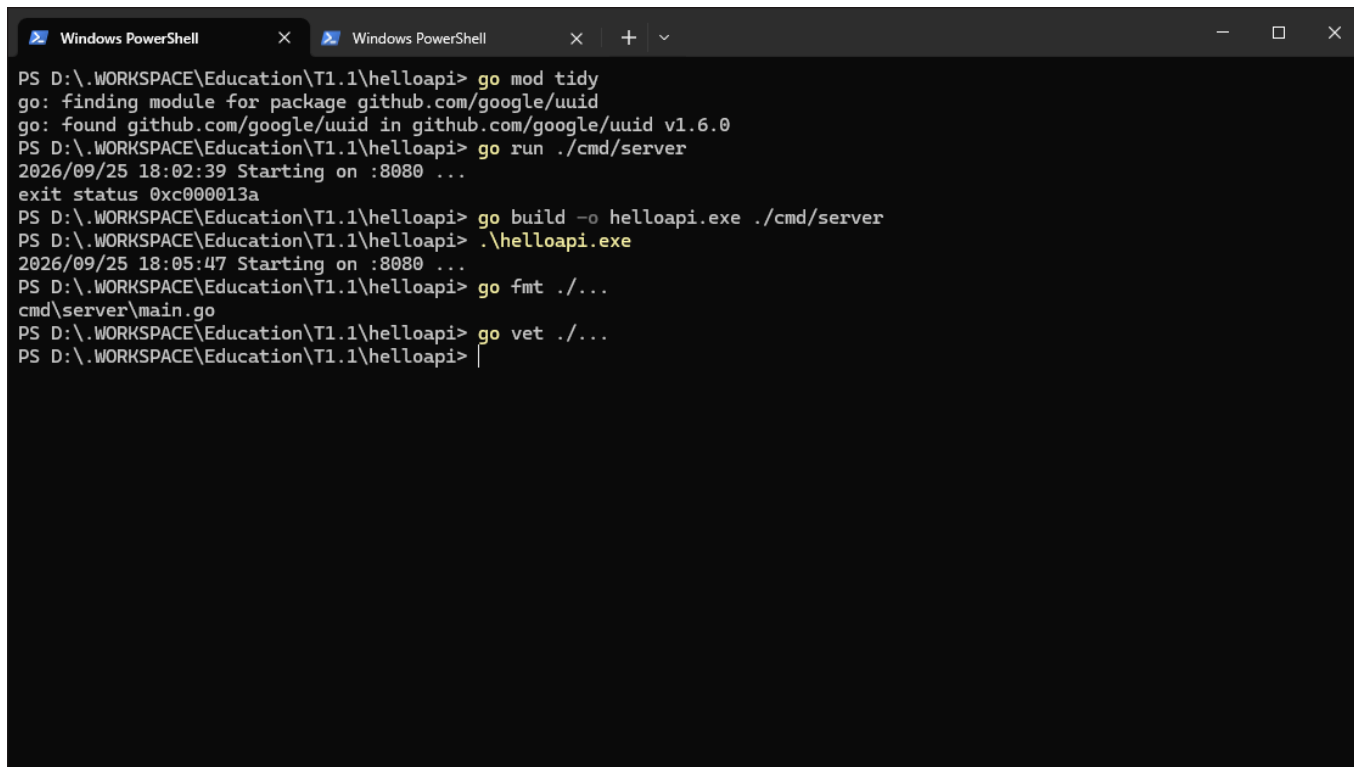
StatusCode      : 200
StatusDescription : OK
Content          : {"id":"b78e46bd-8399-427c-ba8e-9e36617b2769","name":"Gopher"}
RawContent       : HTTP/1.1 200 OK
                  Content-Length: 62
                  Content-Type: application/json
                  Date: Fri, 25 Sep 2026 15:06:40 GMT
                  {"id":"b78e46bd-8399-427c-ba8e-9e36617b2769","name":"Gopher"}
Forms           : {}
Headers         : {[Content-Length, 62], [Content-Type, application/json], [Date, Fri, 25 Sep 2026 15:06:40 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 62

PS C:\Users\Falcion> |
```

Линтер и базовые проверки

Форматируем код и запускаем встроенный анализатор:

```
go fmt ./...
go vet ./...
```



```
Windows PowerShell
PS D:\.WORKSPACE\Education\T1.1\helloapi> go mod tidy
go: finding module for package github.com/google/uuid
go: found github.com/google/uuid in github.com/google/uuid v1.6.0
PS D:\.WORKSPACE\Education\T1.1\helloapi> go run ./cmd/server
2026/09/25 18:02:39 Starting on :8080 ...
exit status 0xc000013a
PS D:\.WORKSPACE\Education\T1.1\helloapi> go build -o helloapi.exe ./cmd/server
PS D:\.WORKSPACE\Education\T1.1\helloapi> .\helloapi.exe
2026/09/25 18:05:47 Starting on :8080 ...
PS D:\.WORKSPACE\Education\T1.1\helloapi> go fmt ./...
cmd/server/main.go
PS D:\.WORKSPACE\Education\T1.1\helloapi> go vet ./...
PS D:\.WORKSPACE\Education\T1.1\helloapi> |
```

Опциональное задание на конфигурацию через ENV и создание сигнала сервера

Доработаем приложение: сделаем порт настраиваемым через переменную `APP_PORT`, добавим маршрут `/health` с временем в формате RFC3339 и простое логирование запросов.

```
package main

import (
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "os"
    "time"
```

```

        "github.com/google/uuid"
    )

    type user struct {
        ID    string `json:"id"`
        Name  string `json:"name"`
    }

    type health struct {
        Status string `json:"status"`
        Time   string `json:"time"`
    }

    func main() {
        port := os.Getenv("APP_PORT")
        if port == "" {
            port = "8080"
        }

        mux := http.NewServeMux()

        mux.HandleFunc("/hello", func(w http.ResponseWriter, r *http.Request)
        {
            log.Printf("%s %s", r.Method, r.URL.Path)
            fmt.Fprintf(w, "Hello, world!")
        })

        mux.HandleFunc("/user", func(w http.ResponseWriter, r *http.Request) {
            log.Printf("%s %s", r.Method, r.URL.Path)
            w.Header().Set("Content-Type", "application/json")
            _ = json.NewEncoder(w).Encode(user{
                ID:    uuid.NewString(),
                Name: "Gopher",
            })
        })

        mux.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request)
        {
            log.Printf("%s %s", r.Method, r.URL.Path)
            w.Header().Set("Content-Type", "application/json")
            _ = json.NewEncoder(w).Encode(health{
                Status: "ok",
                Time:   time.Now().Format(time.RFC3339),
            })
        })
    }

```

```
    addr := ":" + port
    log.Printf("Starting on %s ...", addr)
    log.Fatal(http.ListenAndServe(addr, mux))
}
```

Запуск на другом порту:

```
$env:APP_PORT="8081"
go run ./cmd/server
curl http://localhost:8081/health
```

```
Windows PowerShell x Windows PowerShell + v
PS D:\.WORKSPACE\Education\T1.1\helloapi> $env:APP_PORT="8081"
PS D:\.WORKSPACE\Education\T1.1\helloapi> go run ./cmd/server
2026/09/25 18:10:35 Starting on :8081 ...
|
```

```
Windows PowerShell x Windows PowerShell + v
PS C:\Users\Falcion> curl http://localhost:8081/health

StatusCode      : 200
StatusDescription : OK
Content         : {"status":"ok","time":"2026-09-25T18:10:50+03:00"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 51
                  Content-Type: application/json
                  Date: Fri, 25 Sep 2026 15:10:50 GMT

                  {"status":"ok","time":"2026-09-25T18:10:50+03:00"}

Forms           : {}
Headers         : {[Content-Length, 51], [Content-Type, application/json], [Date, Fri, 25 Sep 2026 15:10:50 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 51

PS C:\Users\Falcion> |
```